

# **Cryptographic Hash Algorithm Laurus**

**Designer: Wenyi Jia<sup>1,2</sup>, Changlun Li<sup>1,2</sup>, Yupeng Liu<sup>1</sup>,  
Honggang Hu<sup>3</sup>, Liang Bai<sup>1,2</sup>, Yitong Liu<sup>1</sup>, Ye Peng<sup>1</sup>, Jiali Wu<sup>1</sup>**

**Completed by: <sup>1</sup>Xingtang Telecommunications Technology  
Co., Ltd. , <sup>2</sup>Beijing Key Laboratory of Post-Quantum  
Cryptography, <sup>3</sup>University of Science and Technology of China**

**Date: 2026-06-29**



## Contents

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>1 Introduction .....</b>                       | <b>1</b>  |
| <b>2 Algorithm Description.....</b>               | <b>1</b>  |
| 2.1 Overview .....                                | 1         |
| 2.1.1 Notation .....                              | 1         |
| 2.1.2 Word (Bit) Order and Conventions .....      | 2         |
| 2.2 Mode of Operation .....                       | 2         |
| 2.2.1 The Laurus Algorithm .....                  | 3         |
| 2.2.2 Padding.....                                | 4         |
| 2.3 Permutation P .....                           | 4         |
| 2.3.1 The State of Permutation P.....             | 4         |
| 2.3.2 Constant Addition $\iota$ .....             | 5         |
| 2.3.3 Nonlinear Transformation $\chi$ .....       | 5         |
| 2.3.4 Linear Transformation $\theta$ .....        | 6         |
| 2.3.5 Transposition Transformation $\pi$ .....    | 7         |
| 2.4 HASH Function .....                           | 7         |
| 2.5 XOF Function .....                            | 7         |
| <b>3 Design Rationale .....</b>                   | <b>8</b>  |
| 3.1 The Design of the Mode of Operation.....      | 8         |
| 3.2 Design of Permutation P.....                  | 9         |
| 3.2.1 Design of Nonlinear Transformation.....     | 10        |
| 3.2.2 Design of Linear Transformation .....       | 12        |
| 3.2.3 Design of Transposition Transformation..... | 13        |
| 3.2.4 Design of Constant Addition .....           | 14        |
| 3.3 Parameter Selection Rationale.....            | 15        |
| <b>4 Security Claims and Analysis .....</b>       | <b>15</b> |

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| 4.1 Theoretical Security .....                                         | 15        |
| 4.1.1 Security Analysis of the Mode of Operation.....                  | 15        |
| 4.1.2 Analysis of the Permutation P .....                              | 15        |
| 4.2 Concrete Security .....                                            | 25        |
| 4.3 Other Security Properties .....                                    | 25        |
| 4.3.1 Statistical Test Analysis .....                                  | 25        |
| 4.3.2 Length Extension Attacks.....                                    | 28        |
| <b>5 Performance Evaluation .....</b>                                  | <b>28</b> |
| 5.1 Performance Analysis.....                                          | 28        |
| 5.1.1 Design Efficiency of the Overall Structure .....                 | 28        |
| 5.1.2 Software and Hardware Efficiency of Core Components.....         | 29        |
| 5.2 Performance Testing .....                                          | 29        |
| 5.2.1 Software Performance Testing .....                               | 29        |
| 5.2.2 Hardware Performance Testing .....                               | 31        |
| <b>6 Features .....</b>                                                | <b>33</b> |
| 6.1 Innovativeness .....                                               | 33        |
| 6.2 Simplicity.....                                                    | 33        |
| 6.3 Flexibility.....                                                   | 34        |
| 6.4 Implementation Performance Advantages on Different Platforms ..... | 34        |
| <b>7 Comparison with SHA-3 Algorithm .....</b>                         | <b>35</b> |
| <b>8 References.....</b>                                               | <b>35</b> |

# 1 Introduction

In response to the call for cryptographic hash algorithm submissions under the Next-generation Commercial Cryptographic Algorithms Program (NGCC) held by the Institute of Commercial Cryptography Standards, we have developed the cryptographic hash algorithm Laurus. Laurus comprises three hash algorithms with output lengths of 512, 768, and 1024 bits, respectively, and one extendable-output function (XOF). Based on the different hash output lengths, the three hash algorithms are named Laurus-512, Laurus-768, and Laurus-1024, respectively, and the extendable-output function is named Laurus-XOF. The specific characteristics of the four algorithms are given in Table 1-1.

Table 1-1 Specific Characteristics of Laurus-512/768/1024/XOF

| Algorithm Name | Input Message Bit Length | Output Hash Bit Length |
|----------------|--------------------------|------------------------|
| Laurus-512     | $\leq 2^{64}-1$          | 512                    |
| Laurus-768     | $\leq 2^{64}-1$          | 768                    |
| Laurus-1024    | $\leq 2^{64}-1$          | 1024                   |
| Laurus-XOF     | $\leq 2^{64}-1$          | $\leq 2^{64}-1$        |

The Laurus hash algorithm is capable of resisting known classical computational attacks and quantum computational attacks. It can be implemented efficiently on a wide range of software and hardware platforms, featuring security, efficiency, and flexibility.

This document provides a description of the algorithm, elaborates its design rationale, presents a preliminary security analysis, and reports performance evaluations.

## 2 Algorithm Description

### 2.1 Overview

The Laurus algorithm adopts a self-designed sponge-like structure, with the internal compression function employing a permutation of state size 1600 bits. Based on the Laurus algorithm, we present the Laurus-512, Laurus-768, and Laurus-1024 hash algorithms, with output lengths of 512, 768, and 1024 bits, respectively, and simultaneously design the extendable-output function Laurus-XOF.

#### 2.1.1 Notation

|          |                                                |
|----------|------------------------------------------------|
| 0x       | Prefix indicating a hexadecimal number.        |
| =        | Assignment operator, may also denote equality. |
| +        | Arithmetic addition operator.                  |
| $\oplus$ | Bitwise XOR operator.                          |

|                   |                                                                                   |
|-------------------|-----------------------------------------------------------------------------------|
| $\&$              | Bitwise AND operator.                                                             |
| $\text{mod}$      | Integer modulo operator.                                                          |
| $\parallel$       | String concatenation operator.                                                    |
| $\#$              | Number of elements in a set.                                                      |
| $\min$            | Returns the minimum value among a set of elements.                                |
| $\max$            | Returns the maximum value among a set of elements.                                |
| $\lll n$          | Operator for cyclic left shift of operand by $n$ bits.                            |
| $\ggg n$          | Operator for cyclic right shift of operand by $n$ bits.                           |
| $\ll n$           | Operator for left shift of operand by $n$ bits, filling lower bits with 0.        |
| $\gg n$           | Operator for right shift of operand by $n$ bits, filling higher bits with 0.      |
| $\{0, 1\}^n$      | Space of $n$ -bit strings.                                                        |
| $A^T$             | Transpose operation of matrix (or vector) $A$ .                                   |
| $\text{MSB}_n(x)$ | The $n$ most significant bits of $x$ .                                            |
| $\text{LSB}_n(x)$ | The $n$ least significant bits of $x$ .                                           |
| $0^n$             | All-zero string of $n$ bits.                                                      |
| $[i]_n$           | The $n$ -bit binary representation string of the non-negative integer $i < 2^n$ . |
| $\lceil \rceil$   | Ceiling operator.                                                                 |
| $N$               | Number of iterations of permutation $P$ , set to 16.                              |
| $\text{RC}_i$     | 64-bit round constant for round $i$ .                                             |

### 2.1.2 Word (Bit) Order and Conventions

Data variables are represented with the most significant bit (or block) on the left and the least significant bit (or block) on the right, numbered from left to right starting from 0. For example, for an  $n$ -bit number, bits from highest to lowest (left to right) are referred to as bit 0, bit 1, bit 2, ..., bit  $n-1$ . When a variable is divided into several blocks, the leftmost (most significant) block consists of the most significant part of the original data, and the rightmost (least significant) block consists of the least significant part of the original data.

Data adopts big-endian byte order. When the input (or output) is a word or byte stream, the lower address corresponds to higher-order data (i.e., left side), and the higher address corresponds to lower-order data (i.e., right side). Every 8 (or 4) bytes form a 64-bit (or 32-bit) word.

Unless otherwise specified, all polynomials, vectors, and matrices in the following text are defined over the binary field.

## 2.2 Mode of Operation

The mode of operation of the Laurus algorithm consists of three phases: **padding**, **absorbing**, and **squeezing**.

Padding Phase: Based on the security level parameter  $c$ , the input message  $M$  is padded to meet the specified length requirements, resulting in the padded message  $M_{\text{pad}} = M_0 || M_1 || \dots || M_l$ .

Absorbing Phase: Starting from the initial state  $S_0$ , the padded message blocks  $M_i$  are iteratively compressed to update the hash state  $S_i$ .

Squeezing Phase: Using the hash state produced in the absorbing phase as input, an extended output is performed according to the required output length  $|h|$  to obtain the hash value  $h = \text{MSB}_{|h|}(h_0 || h_1 || \dots || h_{\lceil |h|/(t+b-c') \rceil - 1})$ .

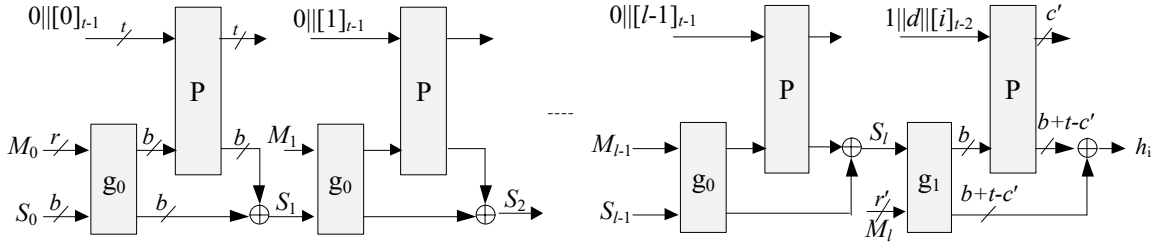


Figure 2-1 Laurus mode of operation

The Laurus mode of operation is shown in Figure 2-1, where  $P$  is a permutation from  $\{0,1\}^{t+b}$  to  $\{0,1\}^{t+b}$ ,  $g_0$  is a function from  $\{0,1\}^{r+b}$  to  $\{0,1\}^{2b}$ , and  $g_1$  is a function from  $\{0,1\}^{r'+b}$  to  $\{0,1\}^{t+2b-c'}$ , with  $b=r+c$ . The variable  $d$  is a 1-bit auxiliary variable determined by the message length. The state  $S_i$  ( $0 \leq i \leq l$ ) has length  $b$  bits. The message blocks  $M_i$  ( $0 \leq i \leq l-1$ ) have length  $r$  bit, and the last block  $M_l$  has length  $r'$  bits, and the output  $h_i$  ( $0 \leq i \leq \lceil |h|/(t+b-c') \rceil - 1$ ) have length  $t+b-c'$  bits.

$$g_0(x) = \text{MSB}_b(x) || (\text{LSB}_r(x) || \text{MSB}_c(x));$$

$$g_1(x) = \text{LSB}_b(x) || \text{LSB}_{t+b-c'}(x).$$

### 2.2.1 The Laurus Algorithm

For the Laurus algorithm, we fix  $t = 64$ ,  $b=1536$ ,  $r'=512$ ,  $c'=512$ . To support different security levels and functional outputs, we use the security parameter  $c$  and the function identifier  $fid$  to control Laurus.  $fid=0$  is used for hash algorithms, and  $fid=1$  is used for the extendable output function (XOF).

The input message  $M$  has bit length  $|M|$ , satisfying  $|M| \leq 2^{64}-1$ . The output hash value  $h$  has bit length  $|h|$ . When  $fid=0$ , it is required that  $|h| \leq 1088$ , when  $fid=1$ , it is required that  $|h| \leq 2^{64}-1$ .

$h = \text{Laurus}[c, fid](M, |h|)$  is computed as follows:

- (1) Compute  $M_0 || M_1 || \dots || M_l = \text{Pad}[c](M)$ ;
- (2)  $S_0 = [2^{63}fid + c]_{1536}$ ;
- (3) For  $1 \leq i \leq l$ , execute:

- $$S_i = \text{LSB}_{1536}(\text{P}(0 \parallel [i-1]_{63} \parallel \text{MSB}_{1536}(\text{g}_0(M_{i-1} \parallel S_{i-1})))) \oplus \text{LSB}_{1536}(\text{g}_0(M_{i-1} \parallel S_{i-1}));$$
- (4) If  $|M| = 0$  or  $(|M|-512) \bmod (1536-c) \neq 0$ , set  $d = 0$ ; otherwise set  $d = 1$ .
- (5) For  $0 \leq i \leq \lceil |h|/1088 \rceil - 1$ , execute:
- $$h_i = \text{LSB}_{1088}(\text{P}(1 \parallel d \parallel [i]_{62} \parallel \text{MSB}_{1536}(\text{g}_1(S_i \parallel M_i)))) \oplus \text{LSB}_{1088}(\text{g}_1(S_i \parallel M_i));$$
- (6) Return  $h = \text{MSB}_{|h|}(h_0 \parallel h_1 \parallel \dots \parallel h_{\lceil |h|/1088 \rceil - 1})$ .

### 2.2.2 Padding

Pad is the message padding function controlled by the security parameter  $c$ .  $M_{\text{pad}} = \text{Pad}[c](M)$  is computed as follows.

If  $|M|=0$  or  $(|M|-512) \bmod r \neq 0$ ,  $M_{\text{pad}} = M_0 \parallel M_1 \parallel \dots \parallel M_l = M \parallel 1 \parallel 0^{r-1 - ((|M|-512) \bmod r)}$ ; Otherwise,  $M_{\text{pad}} = M_0 \parallel M_1 \parallel \dots \parallel M_l = M$ .

where  $|M|$  is the bit length of message  $M$ ,  $r=1536-c$ , each  $M_i$  ( $0 \leq i \leq l-1$ ) has length  $r$  bits,  $M_l$  has length 512 bits, and  $l = \max(0, \lceil (|M|-512)/r \rceil)$ .

## 2.3 Permutation P

The Laurus hash algorithm uses a permutation  $P$  from  $\{0,1\}^{1600}$  to  $\{0,1\}^{1600}$ . Permutation  $P$  adopts an SPN (Substitution-Permutation Network) structure and is formed by iterating the round function  $f$  for  $N$  rounds. The input of permutation  $P$  is a 1600 bit state  $A$ , and the output is a 1600 bit state  $B$ , with  $B=P(A)$  executed as follows:

- (1)  $A_0 = A$ ;
- (2) For  $i = 0$  to  $N-1$ , execute:  
 $A_{i+1} = f(A_i, i_r)$ ;
- (3) Return  $B = A_N$ .

The round function  $f$  takes as input a 1600 bit state  $A_i$  and a round index  $i_r$ .  $f$  consists of four components: the constant addition  $\mathfrak{u}$ , the nonlinear transformation  $\chi$ , the linear transformation  $\theta$ , and the transposition transformation  $\pi$ . Specifically:  $f(A_i, i_r) = \pi(\theta(\chi(\mathfrak{u}(A_i, i_r))))$ .

### 2.3.1 The State of Permutation P

The 1600-bit state of permutation  $P$  is organized as 5 planes of 320 bits each, and each plane consists of 5 lanes of 64 bits each. Thus, the state of  $P$  can be viewed as a  $5 \times 5 \times 64$  cube.

Let  $A$  denotes the entire 1600-bit state. For  $0 \leq i \leq 4$ , let  $A[i]$  denotes the  $i$ -th plane of  $A$ , which is 320 bits. For  $0 \leq j \leq 4$ , let  $A[i][j]$  denotes the  $j$ -th lane of  $A[i]$ , which is 64 bits. For  $0 \leq k \leq 63$ , let  $A[i][j][k]$  denotes the  $k$ -th bit of  $A[i][j]$ , which is 1 bit.

Thus,

$$\begin{aligned}
A &= A[0] \parallel A[1] \parallel \dots \parallel A[4]; \\
A[i] &= A[i][0] \parallel A[i][1] \parallel \dots \parallel A[i][4], \quad 0 \leq i \leq 4; \\
A[i][j] &= A[i][j][0] \parallel A[i][j][1] \parallel \dots \parallel A[i][j][63], \quad 0 \leq i \leq 4, \quad 0 \leq j \leq 4.
\end{aligned}$$

### 2.3.2 Constant Addition $\iota$

The constant addition  $\iota$  adds the 64-bit round constant  $RC_r$  of round  $r$  to the lane  $A[0][0]$ , while keeping all other lanes unchanged.  $B = \iota(A, r)$  is executed as follows:

- (1)  $B = A$ ;
- (2)  $B[0][0] = A[0][0] \oplus RC_r$ ;
- (3) Return  $B$ .

The round constants  $RC_r$  ( $0 \leq r \leq N-1$ ) are computed from constant  $C$ , with specific values listed in Table 2-1. Constant  $C$  is the first 64 bits of the fractional part of the natural constant  $e$ , represented in hexadecimal as:

$$C = 0xb7e151628aed2a6a.$$

$RC_r = 2^r \otimes C$ , where “ $\otimes$ ” denotes multiplication in the finite field  $GF(2^{64}) = GF(2)[x]/(x^{64} + x^{63} + x^{62} + x^{54} + x^{40} + x^{35} + x^{15} + x + 1)$ .

Table 2-1 Round Constants  $RC_r$

| Round $r$ | Round Constant $RC_r$ | Round $r$ | Round Constant $RC_r$ |
|-----------|-----------------------|-----------|-----------------------|
| 0         | 0xb7e151628aed2a6a    | 8         | 0x5dd1971aed536b16    |
| 1         | 0xaf82a3cd15dad4d7    | 9         | 0xbba32e35daa6d62c    |
| 2         | 0x9f4546922bb529ad    | 10        | 0xb7065d63b54d2c5b    |
| 3         | 0xfeca8c2c576ad359    | 11        | 0xae4cbbcf6a9ad8b5    |
| 4         | 0x3dd51950aed526b1    | 12        | 0x9cd97696d5353169    |
| 5         | 0x7baa32a15daa4d62    | 13        | 0xf9f2ec25aa6ae2d1    |
| 6         | 0xf7546542bb549ac4    | 14        | 0x33a5d94354d545a1    |
| 7         | 0x2ee8cb8d76a9b58b    | 15        | 0x674bb286a9aa8b42    |

### 2.3.3 Nonlinear Transformation $\chi$

The nonlinear transformation  $\chi$  updates the state  $A$  by applying 320 identical 5-bit S-boxes in parallel, producing state  $B$ . The lookup table of the S-box is given in Table 2-2, and its circuit implementation is shown in Figure 2-2.

Table 2-2: S-box Lookup Table

| x | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
|---|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|
|---|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|

|      |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| S(x) | 0  | 2  | 4  | 23 | 8  | 10 | 14 | 13 | 16 | 18 | 20 | 15 | 28 | 30 | 26 | 17 |
| x    | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| S(x) | 1  | 19 | 5  | 6  | 9  | 27 | 31 | 12 | 25 | 11 | 29 | 22 | 21 | 7  | 3  | 24 |

The nonlinear transformation  $B=\chi(A)$  is equivalently expressed as:

- (1)  $B[0] = A[0]\&A[2]\&A[3] \oplus A[2]\&A[3]\&A[4] \oplus A[0]\&A[4] \oplus A[3]\&A[4] \oplus A[1]$ ;
- (2)  $B[1] = A[1]\&A[3]\&A[4] \oplus A[0]\&A[1] \oplus A[2]$ ;
- (3)  $B[2] = A[1]\&A[2] \oplus A[3]$ ;
- (4)  $B[3] = A[2]\&A[3] \oplus A[4]$ ;
- (5)  $B[4] = A[3]\&A[4] \oplus A[0]$ ;
- (6) Return B.

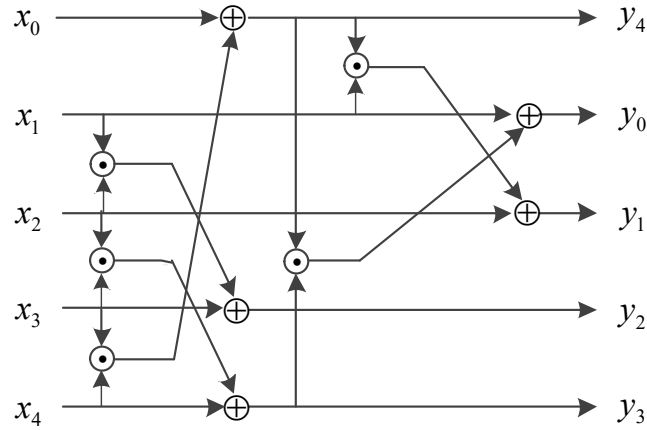


Figure 2-2: Circuit diagram of the S-box

### 2.3.4 Linear Transformation $\theta$

The linear transformation  $\theta$  updates each plane  $A[i]$  of state A separately through the linear transformation  $\theta_i$ , for  $0 \leq i \leq 4$ .  $B=\theta(A)$  is executed as follows:

- (1) For  $0 \leq i \leq 4$ , execute:  
 $B[i] = \theta_i(A[i])$ ;
- (2) Return B.

$\theta_i(0 \leq i \leq 4)$  is a 320-bit linear permutation determined by the parameterized linear permutation Theta. Specifically:

- $$\begin{aligned} \theta_0 &= \text{Theta} [31, 1, 4, 21, 22]; \\ \theta_1 &= \text{Theta} [19, 12, 15, 34, 13]; \\ \theta_2 &= \text{Theta} [28, 35, 58, 9, 23]; \\ \theta_3 &= \text{Theta} [55, 18, 5, 27, 44]; \\ \theta_4 &= \text{Theta} [14, 43, 53, 20, 25]. \end{aligned}$$

$\text{Theta}[t_0, t_1, t_2, t_3, t_4]$  is a 320-bit linear permutation determined by 5 parameters  $(t_0, t_1, t_2, t_3, t_4)$ ,  $Y = \text{Theta}[t_0, t_1, t_2, t_3, t_4](X)$  is executed as follows:

- 1) Divide  $X$  into 5 blocks of 64 bits  $X_i$  ( $0 \leq i \leq 4$ ), i.e.,  $X = X_0 || X_1 || X_2 || X_3 || X_4$ ;
- 2)  $E = X_0 \oplus X_1 \oplus X_2 \oplus X_3 \oplus X_4$ ;
- 3)  $F = (X_0 \lll t_0) \oplus (X_1 \lll t_1) \oplus (X_2 \lll t_2) \oplus (X_3 \lll t_3) \oplus (X_4 \lll t_4)$ ;
- 4) For  $0 \leq i \leq 4$ , execute:  

$$Y_i = X_i \oplus (E \ggg t_i) \oplus F$$
;
- 5)  $Y = Y_0 || Y_1 || Y_2 || Y_3 || Y_4$ .

### 2.3.5 Transposition Transformation $\pi$

The transposition transformation  $\pi$  performs a permutation of the 25 lanes of state  $A$ .  $B = \pi(A)$  is executed as follows:

- (1) For  $0 \leq i \leq 4$  and  $0 \leq j \leq 4$ , execute:  

$$B[i][j] = A[(i+2j) \bmod 5][(3i+3j) \bmod 5]$$
;
- (2) Return  $B$ .

## 2.4 HASH Function

The hash functions Laurus-512, Laurus-768, and Laurus-1024 are constructed based on the Laurus algorithm. Their input is a message  $M$  of length less than  $2^{64}$  bits, computed as follows:

$$\begin{aligned} \text{Laurus-512}(M) &= \text{Laurus}[512, 0](M, 512); \\ \text{Laurus-768}(M) &= \text{Laurus}[768, 0](M, 768); \\ \text{Laurus-1024}(M) &= \text{Laurus}[1024, 0](M, 1024). \end{aligned}$$

## 2.5 XOF Function

The extendable-output function Laurus-XOF is constructed based on the Laurus algorithm. Its input is a message  $M$  of length less than  $2^{64}$  bits, and its output length  $|xof|$  is less than  $2^{64}$  bits, computed as follows:

$$\text{Laurus-XOF}(M, |xof|) = \text{Laurus}[512, 1](M, |xof|).$$

*Note: Since the call for submissions did not specify concrete requirements for extendable-output functions, we provide an example of an XOF here. Its security and efficiency are benchmarked against SHAKE256. Apart from having the same collision resistance, its security properties including (second) preimage resistance and indistinguishability are all stronger than those of SHAKE256. In practice, the parameter  $c$  can be adjusted according to security strength requirements and efficiency needs to give other versions of the extendable-output function.*

## 3 Design Rationale

### 3.1 The Design of the Mode of Operation

Laurus is an iterated hash algorithm. An iterated hash function generally consists of two parts: a mode of operation and an internal function. The internal function is typically a random function or random permutation with fixed input length and fixed output length. The mode of operation transforms it into a hash function that can process inputs of arbitrary length while preserving the corresponding cryptographic properties. In the early stages of hash function design, the security requirements were mainly preimage resistance and collision resistance. With the widespread use of the Random Oracle Model (ROM) in provable security, higher security properties have been demanded of instantiated hash functions, requiring them to behave close to random oracles. Currently, the widely used modes of operation for hash mainly include Merkle-Damgard (MD) and its variants [Mer90, Dam90], Wide-Pipe [Luc05], HAIFA [BD06], and Sponge [BDPV07]. However, these structures have certain disadvantages, which mainly including:

- (1) The classical MD structure cannot achieve the security level of a random oracle, and suffers from generic attacks and length extension attacks. Its variants with post-processing, truncation, and encapsulation have provable security, but the security reduction loss is significant, affecting the overall security strength.
- (2) Structures such as Wide-Pipe and HAIFA have provable security, but they use more parameters that need to be compressed, which affects message processing efficiency. Moreover, in these structures, the internal function is generally assumed to be a random function, but in practice, a random permutation is usually designed first and then converted into a random function, which may incur further reduction loss and also affects efficiency.
- (3) The Sponge structure gained fame through its use in Keccak (SHA-3) [BDPV11, DM15]. The Sponge structure can not only be directly designed based on random permutations, but is also very simple and flexible, and can easily construct various security functions and cryptographic primitives at various security levels. However, the Sponge structure has a significant reduction loss (birthday bound on the security capacity), which reduces the amount of message processed per iteration and affects efficiency.

The mode of operation of the Laurus algorithm is a variant of the VFB structure [LLHL26]. Its design inherits the advantages of the Sponge structure while incorporating the design philosophy of the FWP structure [NP10] to improve the message processing efficiency of the overall structure. The specifics are as follows:

(1) Within the framework of the Sponge structure, using the same random permutation, the security capacity length of the Sponge structure is reduced to increase the amount of message processed per iteration, thereby improving processing efficiency. This allows the Laurus algorithm to achieve a hash output length of 1024 bits without requiring the permutation size to exceed 2048 bits.

(2) Drawing on the design philosophy of the FWP structure, an additional state is introduced to enhance security without increasing the state size of the compression function. The effective length of the iterative state in the Sponge structure is increased, thereby reducing the collision probability of the iterative state and achieving a reduction loss that surpasses the birthday bound of the Sponge structure, thus increasing security strength. At the same time, the random function used in FWP is replaced by a random permutation, making it compatible with current permutation-based design approaches.

(3) To support variable-length output, a parallel post-processing stage is added for the final hash output, processing the iterative state in parallel and improving output efficiency.

Ultimately, the mode of operation of the Laurus algorithm designed according to the above principles, using an internal permutation of the same size and varying the security capacity  $c$ , achieves different output security levels. Compared with the Sponge structure, when the hash output length is 512 bits, the message processing efficiency is improved by nearly a factor of two. Moreover, when used as an extendable output function, the hash output process can be processed in parallel, improving output efficiency and laying the foundation for speed improvements in post-quantum public-key algorithms such as those based on lattices.

## 3.2 Design of Permutation P

The permutation P of the Laurus hash algorithm is designed with a focus on bitsliced implementation and adopts an SPN structure. The round function consists of the constant addition  $\iota$ , the nonlinear transformation  $\chi$ , the linear transformation  $\theta$ , and the transposition transformation  $\pi$ . Each component is designed to be as simple as possible, and through the interplay of these components, a high level of security is achieved with a relatively low number of rounds.

The linear transformation  $\theta$  is the most innovative aspect and design core of permutation P. It is designed based on a parameterized 320-bit linear transformation  $\Theta[t_0, t_1, t_2, t_3, t_4]$ .  $\Theta[t_0, t_1, t_2, t_3, t_4]$  has the same implementation depth as Keccak's linear transformation, but its branch number is increased from 4 to 12. By selecting different parameters, different linear

transformations can be designed. The design of permutation  $P$  revolves around the linear transformation  $\text{Theta}[t_0, t_1, t_2, t_3, t_4]$ , with careful design of other cryptographic components and related parameters.

The design of permutation  $P$  draws inspiration from the Ascon algorithm [DEMS21]. Ascon’s permutation  $P$  divides a 320-bit state into 5 lanes of 64 bits each, using simple linear transformations controlled by different parameters for different lanes to achieve diffusion. The confusion between lanes is provided by a nonlinear transformation (a 5-bit S-box) whose differential and linear branch numbers are both 3, enabling the propagation of differences and linear masks across more lanes.

The permutation  $P$  of Laurus divides the 1600-bit state into 5 planes of 320 bits each, using simple linear transformations controlled by different parameters for different planes to achieve diffusion. The confusion between planes is provided by a nonlinear transformation (a 5-bit S-box). Unlike the design of the S-box in Ascon’s permutation  $P$ , aiming for greater simplicity and efficiency, we did not adopt an S-box with branch number 3; instead, we chose an S-box with branch number 2 and with as few trails as possible. At the same time, we added a transposition transformation  $\pi$  to exchange the 64-bit lanes between different planes, achieving faster inter-plane diffusion. Moreover, the transposition transformation  $\pi$  incurs no additional cost in either software or hardware implementation.

Through careful parameter selection, the permutation  $P$  of Laurus achieves full diffusion in 3 rounds and possesses strong resistance against differential and linear attacks.

Below, we describe in detail the design principles and cryptographic properties of each component of the round function.

### 3.2.1 Design of Nonlinear Transformation

The nonlinear transformation  $\chi$  mainly provides confusion across the corresponding bits of the five planes of state  $A$ . Based on a trade-off between security and efficiency considerations, the design of the nonlinear transformation took into account the algebraic degree of the S-box, the maximum differential probability, the maximum linear bias, and the diffusion properties of differentials and linear masks. At the same time, to enable efficient bitsliced implementation, the S-box must have a compact algebraic expression. Specifically, the properties of the S-box used in the nonlinear transformation  $\chi$  are as follows:

- (1) The algebraic degree is 3; the inverse S-box also has algebraic degree 3;

(2) Maximum differential probability is  $2^{-2}$ ;

(3) Maximum linear bias is  $2^{-2}$ ;

(4) The differential branch number is 2. In the difference distribution table (see Table 3-1), the number of entries where both input and output have Hamming weight 1 is 5, i.e., the number of points with differential branch number 2 is only 5;

(5) The linear branch number is 2. In the linear approximation table (see Table 3-2), the number of entries where both input and output have Hamming weight 1 is 5, i.e., the number of points with linear branch number 2 is only 5;

(6) The S-box can be implemented with only 5 AND gates and 5 XOR gates; see Section 5.2.1 “Software Performance Testing” for details.

To enhance resistance against algebraic attacks, the algebraic degree of the S-box is required to be no less than 3. Since the linear transformation  $\theta$  of permutation P operates on each plane of state A, while the nonlinear transformation  $\chi$  mainly provides confusion across the corresponding bits of the five planes, if the S-box has good diffusion properties, it can enhance the resistance of permutation P against differential and linear attacks. However, if both the differential and linear branch numbers of the S-box reached 3, the complexity of the S-box expression would increase significantly, incurring a high implementation cost. As a compromise, although the designed S-box does not achieve branch number 3, it has as few points as possible with branch number 2.

Table 3-1 Difference Distribution Table :  $DDT[\alpha, \beta] = \#\{x | S(x \oplus \alpha) \oplus S(x) = \beta\}$

|    | 0  | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |   |
|----|----|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|
| 0  | 32 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |   |
| 1  | 0  | 0 | 8 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 0 |
| 2  | 0  | 0 | 0 | 0 | 8 | 0 | 4 | 2 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 2 |
| 3  | 0  | 0 | 0 | 0 | 8 | 4 | 4 | 2 | 0 | 0 | 0  | 0  | 0  | 4  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 2 |
| 4  | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8 | 0 | 4  | 0  | 8  | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 0 |
| 5  | 0  | 0 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 4  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 4  | 0  | 0  | 0  | 4  | 0  | 0 |
| 6  | 0  | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 4 | 0 | 2  | 0  | 4  | 0  | 2  | 2  | 0  | 4  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 2  | 0  | 0  | 4  | 2  | 2 |
| 7  | 0  | 4 | 0 | 2 | 0 | 0 | 0 | 0 | 4 | 0 | 2  | 0  | 4  | 4  | 2  | 2  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 2  | 2 |
| 8  | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0 |
| 9  | 0  | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 4  | 2  | 0  | 0  | 4  | 2  | 0  | 0  | 4  | 2  | 0  | 0  | 4  | 2  | 0  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 2 |
| 10 | 0  | 0 | 0 | 2 | 0 | 4 | 0 | 0 | 0 | 0 | 4  | 2  | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 2  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 2  | 4  | 0  | 0  | 0 |
| 11 | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 0 | 0 | 0  | 0  | 0  | 0  | 4  | 2  | 4  | 4  | 0  | 0  | 0  | 0  | 4  | 2  | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 2 |
| 12 | 0  | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 4  | 0  | 4  | 0  | 0 |
| 13 | 0  | 2 | 4 | 0 | 0 | 2 | 4 | 0 | 0 | 2 | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 2  | 4  | 0  | 0  | 2  | 4  | 0  | 0 |
| 14 | 0  | 2 | 2 | 0 | 0 | 2 | 2 | 0 | 2 | 0 | 2  | 0  | 2  | 0  | 0  | 0  | 2  | 0  | 0  | 2  | 2  | 0  | 0  | 0  | 2  | 0  | 2  | 0  | 2  | 0  | 2  | 0  | 2 |
| 15 | 0  | 0 | 2 | 0 | 0 | 0 | 2 | 2 | 0 | 0 | 0  | 2  | 0  | 0  | 0  | 0  | 2  | 2  | 0  | 2  | 2  | 2  | 0  | 0  | 2  | 2  | 2  | 0  | 2  | 2  | 2  | 2  | 2 |
| 16 | 0  | 8 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0 |
| 17 | 0  | 0 | 8 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 0  | 0 |
| 18 | 0  | 0 | 0 | 0 | 8 | 4 | 4 | 2 | 0 | 0 | 0  | 0  | 0  | 4  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 2 |
| 19 | 0  | 0 | 0 | 0 | 8 | 0 | 4 | 2 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 2  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 2  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 2 |
| 20 | 0  | 0 | 0 | 0 | 0 | 4 | 0 | 4 | 0 | 4 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | 0 |
| 21 | 0  | 0 | 0 | 0 | 0 | 0 | 0 | 4 | 8 | 0 | 0  | 4  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 4  | 0  | 0  | 0  | 0  | 0 |

|    |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 22 | 0 | 4 | 0 | 2 | 0 | 0 | 0 | 0 | 4 | 0 | 2 | 0 | 4 | 4 | 2 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 2 |
| 23 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 4 | 0 | 2 | 0 | 4 | 0 | 2 | 2 | 0 | 4 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 0 | 0 | 4 | 2 | 2 |
| 24 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 | 0 | 4 | 0 | 0 |
| 25 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 4 | 2 | 0 | 0 | 4 | 2 | 0 | 0 | 4 | 2 | 0 | 0 | 4 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 |
| 26 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 4 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 4 | 2 | 4 | 4 | 0 | 0 | 0 | 0 | 0 | 2 | 4 | 4 | 0 | 0 |
| 27 | 0 | 4 | 0 | 0 | 0 | 0 | 0 | 2 | 0 | 4 | 0 | 0 | 0 | 0 | 4 | 2 | 4 | 0 | 0 | 0 | 0 | 0 | 4 | 2 | 4 | 0 | 0 | 0 | 0 | 0 | 0 | 2 |
| 28 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 0 | 2 |
| 29 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 4 | 0 | 0 | 2 | 4 | 0 | 0 | 2 | 4 | 0 | 0 | 2 | 4 | 0 | 0 | 2 |
| 30 | 0 | 0 | 2 | 2 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 2 | 2 | 0 | 0 | 2 | 2 | 0 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 0 |
| 31 | 0 | 2 | 2 | 2 | 0 | 2 | 2 | 0 | 0 | 2 | 0 | 0 | 0 | 2 | 0 | 2 | 2 | 0 | 0 | 0 | 2 | 0 | 0 | 2 | 2 | 0 | 2 | 2 | 2 | 0 | 2 | 0 |

Table 3-2 Linear Approximation Table :  $LAT[\alpha, \beta] = \#\{x | \alpha^T \cdot x = \beta^T \cdot S(x)\} - 16$

|    | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0  | 16 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| 1  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 0  | 0  | -4 | 4  | 0  | 0  | 0  | 0  | 4  | -4 | 4  | 4  | 4  | -4 | 0  | 0  | 0  | 0  | -4 | -4 |
| 2  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 | 0  | 0  | 0  | 0  | -4 | -4 | 4  | -4 |
| 3  | 0  | 0  | 8  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | -4 | -4 | 0  | 0  | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | -4 | 4  | 0  | 0  | 4  | -4 | 0  | 0  |
| 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | -8 | 0  | 0  | 0  | 0  |
| 5  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 0  | 0  | 4  | -4 | 0  | 0  | 0  | 0  | -4 | 4  | -4 | -4 | 4  | -4 | 0  | 0  | 0  | 0  | 4  | 4  |
| 6  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | -4 | -4 | -4 | 4  | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 |
| 7  | 0  | 0  | -8 | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 4  | 4  | 0  | 0  | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | -4 | 0  | 0  | 4  | -4 | 0  | 0  |
| 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 8  | 8  | -8 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| 9  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 4  | -4 | 0  | 0  | -4 | -4 | 4  | -4 | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | -4 | 4  | 0  | 0  |
| 10 | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | -8 | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 | 0  | 0  | 0  | 0  | 4  | 4  | -4 | 4  |
| 11 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | -4 | 4  | 0  | 0  | 4  | -4 | 4  | -4 | 4  | 4  | 4  | -4 | 4  | 4  | 0  | 0  | -4 | -4 | 0  | 0  | 4  | 4  |
| 12 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | -8 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 8  | 0  | 0  | 0  | 0  | 0  |
| 13 | 0  | 0  | 0  | 0  | 0  | 0  | -8 | 0  | 0  | 0  | 4  | -4 | 0  | 0  | 4  | 4  | 4  | -4 | 4  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 | 0  | 0  |
| 14 | 0  | 0  | 0  | 0  | -8 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 8  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 | 0  | 0  | 0  | 0  | 4  | 4  | 4  | -4 |
| 15 | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | -4 | 0  | 0  | 4  | -4 | -4 | 4  | -4 | -4 | 4  | -4 | 4  | 4  | 0  | 0  | 4  | 4  | 0  | 0  | 4  | 4  |
| 16 | 0  | 8  | 0  | 0  | 0  | 4  | 0  | -4 | 0  | 0  | 4  | 4  | 4  | 0  | 0  | -4 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | -4 | 4  | 0  | 0  | 0  | -4 | -4 | 0  |
| 17 | 0  | 8  | 0  | 8  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | -4 |
| 18 | 0  | 8  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | -4 | -4 | 4  | 0  | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | -4 | 0  | 4  | -4 | 0  | 0  | -4 | 0  | 0  | 4  |
| 19 | 0  | -8 | 0  | 8  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | 0  |
| 20 | 0  | 0  | 0  | 8  | 0  | 4  | 0  | -4 | 4  | 4  | 0  | 0  | 0  | 4  | -4 | 0  | 0  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | 0  | 0  | -4 | 4  | 4  | 0  | 0  | 4  |
| 21 | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 4  | 4  | 4  | 4  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | -4 | 0  | -4 | 4  | -4 | 4  | -4 | 0  | 4  | 0  |
| 22 | 0  | 0  | 0  | -8 | 0  | 4  | 0  | 4  | 4  | 4  | 0  | 0  | 0  | 4  | 4  | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | -4 | 4  | 0  | 4  | -4 | 0  |
| 23 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | -4 | -4 | 4  | 4  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | -4 | 4  | 4  | -4 | 0  | 4  | 0  | 4  |
| 24 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | -4 | 4  | -4 | 0  | 0  | -4 | 4  | 4  | -4 | 4  | 4  | 0  | -4 | 0  | 0  | 0  | 4  | 4  | 4  | 0  | 0  | -4 |
| 25 | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | -4 | 0  | 0  | 8  | 0  | 8  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  |
| 26 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | -4 | 0  | 0  | 4  | -4 | -4 | 0  | 0  | 4  | 4  | 4  | -4 | 4  | 0  | 4  | 0  | 4  | 0  | 0  | -4 | -4 | 0  | 4  | 4  | 0  |
| 27 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | 0  | -8 | 0  | 8  | 0  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | 0  |
| 28 | 0  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | -4 | 4  | 0  | 0  | 0  | 4  | 4  | 0  | -4 | 4  | 4  | 4  | 4  | 0  | -4 | 0  | 4  | 4  | 0  | 0  | 0  | 4  | -4 | 0  |
| 29 | 0  | 0  | 0  | 0  | 0  | 4  | 0  | -4 | -4 | 4  | -4 | 4  | 0  | -4 | 0  | 4  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | 0  | 4  | 4  | 4  | 4  | -4 | 0  | 4  | 0  |
| 30 | 0  | 0  | 0  | 0  | 0  | -4 | 0  | 4  | -4 | 4  | 0  | 0  | 0  | 4  | -4 | 0  | 4  | -4 | -4 | -4 | 0  | 4  | 0  | 4  | 4  | 4  | 0  | 0  | 4  | 0  | 0  | 4  |
| 31 | 0  | 0  | 0  | 0  | 0  | -4 | 0  | -4 | 4  | -4 | -4 | 4  | 0  | 4  | 0  | 4  | 0  | 0  | 0  | 0  | 4  | 0  | 4  | -4 | -4 | 4  | 4  | 0  | 4  | 0  | 4  | 0  |

### 3.2.2 Design of Linear Transformation

The linear transformation  $\theta$  updates each plane  $A[i]$  of state  $A$  separately through the linear transformation  $\theta_i$ , for  $0 \leq i \leq 4$ . Each linear transformation  $\theta_i$  is a 320-bit linear permutation determined by the parameterized linear permutation  $\text{Theta}[t_0, t_1, t_2, t_3, t_4]$ , where  $t_i$  are integers from 0 to 63.

$V = \text{Theta}[t_0, t_1, t_2, t_3, t_4](U)$  is a 320-bit linear permutation controlled by parameters  $(t_0, t_1, t_2, t_3, t_4)$ . The 320-bit input is divided into 5 blocks of 64 bits, i.e.,  $U = U_0 || U_1 || U_2 || U_3 || U_4$ ,  $V = V_0 || V_1 || V_2 || V_3 || V_4$ . Using the polynomial representation over the ring  $\text{GF}(2)[x]/(x^{64}+1)$ , it can be expressed as:

$$V_i = U_i + \sum_{j=0}^4 (x^{-t_i} + x^{t_j})U_j, 0 \leq i \leq 4$$

Theta satisfies the following properties:

(1) Symmetry:  $\text{Theta}^T = \text{Theta}$ ;

(2) Invertibility: its inverse transformation is

$$U_i = V_i + (\sum_{k=0}^4 x^{-t_k} + \sum_{k=0}^4 x^{t_k} + \sum_{k=0}^4 x^{-t_k} \sum_{k=0}^4 x^{t_k})^{-1} \sum_{j=0}^4 (1 + (1 + \sum_{k=0}^4 x^{t_k})x^{-t_i} + (1 + \sum_{k=0}^4 x^{-t_k})x^{t_j} + x^{-t_i+t_j})V_j$$

(3) When specific parameters are selected, both differential branch number and linear branch number can reach 12. Each  $\theta_i$  achieves a differential branch number and a linear branch number of 12.

To achieve branch number 12, the parameter selection in  $\text{Theta}[t_0, t_1, t_2, t_3, t_4]$  must satisfy the following requirements:

- ①  $2t_i \neq 2t_j \pmod{64}$ , for arbitrary distinct  $i, j$ , and  $0 \leq i, j \leq 4$ ;
- ②  $t_i \neq -t_j \pmod{64}$ , for arbitrary  $i, j$ , and  $0 \leq i, j \leq 4$ ;
- ③  $t_i + t_j \neq 2t_k \pmod{64}$ , for arbitrary distinct  $i, j, k$ , and  $0 \leq i, j, k \leq 4$ ;
- ④  $t_i + t_j \neq t_k + t_l \pmod{64}$ , for arbitrary distinct  $i, j, k, l$ , and  $0 \leq i, j, k, l \leq 4$ .

The hardware implementation depth of Theta is the same as that of Keccak's linear transformation. The software implementation requires only 18 64-bit XOR operations and 10 cyclic shifts, while the branch number is increased from 4 to 12, significantly enhancing the resistance of permutation P against differential, linear, and other attacks. The STCPM scheme proposed in [HWL24] can also achieve a differential and linear branch number of 12, but its computational depth and complexity are higher than those of the Theta transformation.

### 3.2.3 Design of Transposition Transformation

The primary function of the transposition transformation  $\pi$  is to perform a permutation of the lanes, ensuring that the five lanes on the same plane of state A, after the transposition transformation  $\pi$ , reside in five different planes, thereby achieving rapid diffusion. At the same time, to preserve the branch number of the linear transformation  $\theta_i$ , any two lanes from the same plane of state A, after the transposition transformation  $\pi$ , must not be located in the same column, i.e., no two lanes shall have corresponding bits processed by the same

S-box.

The lane position transformation in transposition transformation  $\pi$  can be viewed as the following matrix operation over GF(5). The index  $(i,j)$  of a lane, after  $\pi$  transformation, becomes  $(i',j')$ . The computation is:

$$\begin{pmatrix} i' \\ j' \end{pmatrix} = M \begin{pmatrix} i \\ j \end{pmatrix} = \begin{pmatrix} 4 & 4 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} i \\ j \end{pmatrix}$$

Transposition transformation  $\pi$  satisfies the following properties:

(1) M is invertible, with the inverse matrix  $M^{-1} = \begin{pmatrix} 1 & 2 \\ 3 & 3 \end{pmatrix}$ .

(2) The transposition transformation  $\pi$  corresponds to lane position transformation matrix  $\begin{pmatrix} 1 & 2 \\ 3 & 3 \end{pmatrix}$ .

(3) The order of matrix M is 24, i.e.,  $\begin{pmatrix} 4 & 4 \\ 1 & 3 \end{pmatrix}^{24} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ . 24 is the maximum order of  $2 \times 2$  invertible matrices with elements in GF(5).

(4) Any two points in the same row, after  $\pi$  transformation, cannot end up in the same row or same column. That is, for any two points with the same horizontal coordinate but different vertical coordinate, after  $\pi$  transformation, both horizontal and vertical coordinate are different. This property is easily verified: assume two arbitrary points with same horizontal coordinate but different vertical coordinate are  $(u,v)$  and  $(u,v')$ ; after  $\pi$  transformation, the coordinate become  $(4u+4v, u+3v)$  and  $(4u+4v', u+3v')$ , from which it follows that  $4u+4v \neq 4u+4v'$  and  $u+3v \neq u+3v'$ .

(5) Any two points in the same column, after  $\pi$  transformation, cannot end up in the same row or same column. That is, for any two points with different horizontal coordinate but same vertical coordinate, after  $\pi$  transformation, both horizontal and vertical coordinate are different. This property is easily verified: assume two arbitrary points with different horizontal coordinate but same vertical coordinate are  $(u,v)$  and  $(u',v)$ ; after  $\pi$  transformation, the coordinate become  $(4u+4v, u+3v)$  and  $(4u'+4v, u'+3v)$ , from which it follows that  $4u+4v \neq 4u'+4v$  and  $u+3v \neq u'+3v$ .

### 3.2.4 Design of Constant Addition

Round constants are introduced in each round primarily to resist slide attacks, rotational attacks, and the like. The constants are taken from the fractional part of the natural constant e, demonstrating that the constant selection contains no hidden backdoors. The round constants are updated by multiplication in the finite field GF( $2^{64}$ ) defined by the irreducible polynomial  $x^{64}+x^{63}+x^{62}+x^{54}+x^{40}+x^{35}+x^{15}+x+1$ , which is different from the operations of the

linear transformation  $\theta$  (performed in the ring  $\text{GF}(2)[x]/(x^{64}+1)$ ), thereby avoiding related attacks that could arise from combining the two.

Round constants use only 64 bits per round, balancing implementation efficiency and resource consumption.

### 3.3 Parameter Selection Rationale

We selected the corresponding parameters based on the properties required for the various cryptographic components mentioned above, and in conjunction with security analysis, further screened from the parameters that satisfy these properties those with strongest resistance against differential, linear, and other attacks.

## 4 Security Claims and Analysis

### 4.1 Theoretical Security

#### 4.1.1 Security Analysis of the Mode of Operation

Assume that in the mode of operation of the Laurus algorithm, the capacity is  $c$ , the hash output length is  $|h|$  bits. Then, the complexity of resisting various attacks for Laurus is as follows:

- (1) Collision attack:  $\min(O(2^{|h|/2}), O(2^{c/2}))$ ;
- (2) Preimage attack:  $\min(O(2^{|h|}), O(2^c))$ ;
- (3) Second preimage attack:  $\min(O(2^{|h|}), O(2^c))$ ;

(4) When  $c \geq 512$ , if the permutation  $P$  is a random permutation, even if the adversary is allowed to make arbitrary queries to this permutation, the complexity of distinguishing the Laurus function from a random oracle is no less than  $O(2^{512})$ . For the detailed proof of the above conclusions, see the document “Laurus Provable Security Analysis.pdf”.

#### 4.1.2 Analysis of the Permutation $P$

##### 4.1.2.1 Differential Analysis

Differential analysis is the most important analysis for permutation  $P$ , and it is also a crucial basis for our parameter selection of the linear transformation  $\theta_i$  and the design of the transposition transformation  $\pi$ .

Since the constant addition  $\iota$  does not need to be considered in differential analysis, permutation  $P$  can be analyzed by considering only the round function  $f$  composed of  $\pi \circ \theta \circ \chi$ . For the  $i$ -th round, let the input difference of the  $\chi$

transformation be  $\Delta X_i$ , the input difference of the  $\theta$  transformation be  $\Delta Y_i$  (i.e., the output difference of  $\chi$ ), and the input difference of the  $\pi$  transformation be  $\Delta Z_i$  (i.e., the output difference of  $\theta$ ). That is:

$$\Delta X_i \xrightarrow{\chi} \Delta Y_i \xrightarrow{\theta} \Delta Z_i \xrightarrow{\pi} \Delta X_{i+1}.$$

### (1) 2-Round Differential Analysis

Since all  $\theta_i$  transformations have a differential branch number of 12, and after the  $\pi$  transformation any two lanes from the same plane are not in the same column, it is easy to prove that the minimum number of active S-boxes for a 2-round differential of permutation P is 12. Through computer search, it can similarly be obtained that the minimum number of active S-boxes for a 2-round differential of permutation P is 12.

### (2) 3-Round Differential Analysis

For 3-round differential analysis, we use an MILP approach to search for active S-boxes. The S-box is characterized by its difference distribution table, and the linear transformation is characterized by bitwise XOR. Direct 3-round search is very inefficient; we use certain techniques to search for lower and upper bounds on the minimum number of 3-round differential active S-boxes.

We primarily obtain a lower bound for the 3-round minimum differential active S-boxes through a detailed 2-round differential active S-box search. Specifically, we search for the minimum number of 2-round differential active S-boxes with input  $\Delta Y_i$  and output  $\Delta X_{i+1}$ :

$$\Delta Y_i \xrightarrow{\theta} \Delta Z_i \xrightarrow{\pi} \Delta X_{i+1}$$

For convenience, denote  $\Delta Y_{i,j}$  as the  $j$ -th plane of  $\Delta Y_i$ . We consider 32 cases based on whether  $\Delta Y_{i,j}$  is zero. For these 32 cases, we compute the number of active S-boxes for 2 rounds, as shown in Table 4-1, where 0 indicates  $\Delta Y_{i,j}=0$  and 1 indicates  $\Delta Y_{i,j}\neq 0$ . Due to high complexity, when the number of non-zero input difference planes in  $\Delta Y_i$  exceeds 3, we cannot obtain the minimum number of 2-round differential active S-boxes; using Theorem 4-1, we provide a conservative lower bound.

Table 4-1 Detailed Values of 2-Round Differential Active S-Boxes

| $\Delta Y_{i,0}, \Delta Y_{i,1}, \Delta Y_{i,2},$<br>$\Delta Y_{i,3}, \Delta Y_{i,4}$ | Active S-boxes | $\Delta Y_{i,0}, \Delta Y_{i,1}, \Delta Y_{i,2},$<br>$\Delta Y_{i,3}, \Delta Y_{i,4}$ | Active S-boxes |
|---------------------------------------------------------------------------------------|----------------|---------------------------------------------------------------------------------------|----------------|
| 0,0,0,0,0                                                                             | 0              | 0,0,1,1,1                                                                             | 23             |
| 0,0,0,0,1                                                                             | 12             | 0,1,0,1,1                                                                             | 24             |
| 0,0,0,1,0                                                                             | 12             | 0,1,1,0,1                                                                             | 23             |
| 0,0,1,0,0                                                                             | 12             | 0,1,1,1,0                                                                             | 23             |

|           |    |           |           |
|-----------|----|-----------|-----------|
| 0,1,0,0,0 | 12 | 1,0,0,1,1 | 23        |
| 1,0,0,0,0 | 12 | 1,0,1,0,1 | 23        |
| 0,0,0,1,1 | 18 | 1,0,1,1,0 | 23        |
| 0,0,1,0,1 | 18 | 1,1,0,0,1 | 24        |
| 1,0,0,1,0 | 18 | 1,1,0,1,0 | 24        |
| 0,1,0,0,1 | 18 | 1,1,1,0,0 | 24        |
| 0,1,0,1,0 | 18 | 0,1,1,1,1 | $\geq 24$ |
| 0,1,1,0,0 | 18 | 1,0,1,1,1 | $\geq 24$ |
| 1,0,0,0,1 | 18 | 1,1,0,1,1 | $\geq 24$ |
| 0,0,1,1,0 | 18 | 1,1,1,0,1 | $\geq 24$ |
| 1,0,1,0,0 | 18 | 1,1,1,1,0 | $\geq 24$ |
| 1,1,0,0,0 | 18 | 1,1,1,1,1 | $\geq 24$ |

From Table 4-1, it can be seen that when the number of non-zero input difference planes in  $Y_i$  exceeds 2, the minimum value of 2-round differential active S-boxes is 23.

**Theorem 4-1:** Assuming  $\Delta Y_i$  originally has  $t$  planes with non-zero input difference, then after adding one more plane with non-zero input difference, the minimum number of 2-round differential active S-boxes will not decrease.

*Proof:* Assume  $\Delta Y_i$  with one additional plane's input difference added is  $\Delta Y'_i$ , and let  $\Delta X_{i+1} = \pi(\theta(Y_i))$ ,  $\Delta X'_{i+1} = \pi(\theta(Y'_i))$ . Since  $\Delta Y'_i$  has one additional plane's input differential mask compared to  $\Delta Y_i$ , according to the properties of  $\pi$  and  $\theta$ ,  $\Delta X'_{i+1}$  has additional non-zero lanes compared to  $\Delta X_{i+1}$ . Therefore, the number of active S-boxes for  $\Delta Y'_i$  is no less than that for  $\Delta Y_i$ , and the number of active S-boxes for  $\Delta X'_{i+1}$  is no less than that for  $\Delta X_{i+1}$ . Thus, the minimum number of 2-round differential active S-boxes will not decrease. ■

We now prove that when the number of non-zero input difference planes in round  $i$  is less than 3, the minimum number of differential active S-boxes over rounds  $i$ ,  $i+1$ , and  $i+2$  is at least 24. Since direct search is too time-consuming, we perform the following 2-round search:

$$\Delta Y_i \xrightarrow{\theta} \Delta Z_i \xrightarrow{\pi} \Delta X_{i+1} \xrightarrow{\chi} \Delta Y_{i+1}$$

We search for differential pairs with input  $\Delta Y_i$  and output  $\Delta Y_{i+1}$ . To obtain refined search results, we exhaustively search all possible values where the plane weights of  $\Delta Y_i$  and  $\Delta Y_{i+1}$  are 1 and 2 (we define the plane weight of  $\Delta Y_i$  as the number of planes with non-zero differential in  $\Delta Y_i$ ), totaling  $15 \times 15 = 225$  cases. It can be computed that the minimum active S-boxes for these 225 cases are all no less than 23. In fact, due to time constraints, we stopped searching when the lower bound reached 23; actual values may exceed 23. When a real differential path exists, the minimum number of differential active S-boxes we searched for is 26, at which point the plane weight of  $\Delta Y_i$  is 1 and the plane

weight of  $\Delta Y_{i+1}$  is 2.

From the above analysis, when the plane weight of  $\Delta Y_i$  is at most 2 and that of  $\Delta Y_{i+1}$  is at most 2, the number of active S-boxes over three consecutive rounds is at least 24.

In summary, considering the following three cases, the minimum number of active S-boxes over three consecutive rounds of permutation P is no less than 24.

1)When the plane weight of  $\Delta Y_i$  is greater than 2, from Table 4-1, the number of active S-boxes in rounds  $i$  and  $i+1$  is no less than 23, so over three rounds it is no less than 24.

2)When the plane weight of  $\Delta Y_i$  is at most 2 and the plane weight of  $\Delta Y_{i+1}$  is greater than 2, from Table 4-1, the number of active S-boxes in rounds  $i+1$  and  $i+2$  is no less than 23, so over three rounds it is no less than 24.

3)When the plane weights of  $\Delta Y_i$  and  $\Delta Y_{i+1}$  are both at most 2, then the number of active S-boxes over three consecutive rounds is at least 24.

In fact, for 3-round differential active S-boxes, the calculation of the above 24 active S-boxes is very conservative. We performed an MILP search over rounds  $i, i+1, i+2$ . After many attempts and prolonged searching, we found that when  $\Delta Y_i$  has only one plane with non-zero input difference and the input and output differences of that plane are both 6, the minimum number of active S-boxes for a real 3-round path is 64, i.e., the upper bound for the number of 3-round differential active S-boxes is 64, as shown in Table 4-2. **We conjecture that 64 is very likely the lower bound for the number of 3-round differential active S-boxes.**

Table 4-2: Detailed Values of 3-Round Differential Active S-Boxes

| $\Delta Y_{i,0}, \Delta Y_{i,1}, \Delta Y_{i,2}, \Delta Y_{i,3}, \Delta Y_{i,4}$ | Active S-boxes | Active S-boxes per Round |
|----------------------------------------------------------------------------------|----------------|--------------------------|
| 1,0,0,0,0                                                                        | 65             | 6---6---53               |
| 0,1,0,0,0                                                                        | 66             | 6---6---54               |
| 0,0,1,0,0                                                                        | 66             | 6---6---54               |
| 0,0,0,1,0                                                                        | 64             | 6---6---52               |
| 0,0,0,0,1                                                                        | 69             | 6---6---57               |

In summary, the number of differential active S-boxes for permutation P is shown in Table 4-3.

Table 4-3: Number of Differential Active S-boxes for Permutation P

| Rounds                          | 1 | 2  | 3       |
|---------------------------------|---|----|---------|
| Min Differential Active S-boxes | 1 | 12 | [24,64] |

#### 4.1.2.2 Linear Analysis

Since the constant addition  $\iota$  not need to be considered in linear analysis, permutation P can be analyzed by considering only the round function  $f$  composed of  $\pi \circ \theta \circ \chi$ . For linear masks, the computation can be expressed as  $\Gamma X_{i+1} = \theta^T(\pi^T(\chi^{-1}(\Gamma X_i)))$ . From Sections 3.2.2 and 3.2.3, we have  $\theta^T = \theta$  and  $\pi^T = \pi^{-1}$ , so  $\Gamma X_{i+1} = \theta(\pi^{-1}(\chi^{-1}(\Gamma X_i)))$ . For the  $i$ -th round, let the input mask of the  $\chi^{-1}$  transformation be  $\Gamma X_i$ , the input mask of the  $\pi^{-1}$  transformation be  $\Gamma Z_i$  (i.e., the output mask of  $\chi^{-1}$ ), and the input mask of the  $\theta$  transformation be  $\Gamma Y_i$  (i.e., the output mask of  $\pi^{-1}$ ). That is:

$$\Gamma X_i \xrightarrow{\chi^{-1}} \Gamma Z_i \xrightarrow{\pi^{-1}} \Gamma Y_i \xrightarrow{\theta} \Gamma X_{i+1}$$

##### (1) 2-Round Linear Analysis

Since all  $\theta_i$  transformations have linear branch number 12, and after  $\pi^{-1}$  transformation, any two lanes from the same plane are not in the same column, it is easy to prove that the minimum number of active S-boxes for 2-round linear analysis of permutation P is 12. Through computer search, it can similarly be obtained that the minimum number of active S-boxes for 2-round linear analysis of permutation P is 12.

##### (2) 3-Round Linear Analysis

For 3-round linear analysis, we use MILP methods to search for linear active S-boxes. The S-box is characterized using its linear approximation table and converted to inequalities; linear transformation is characterized using bitwise XOR. Direct 3-round search is very inefficient; we use certain techniques to search for lower and upper bounds on the minimum number of 3-round linear active S-boxes.

We primarily obtain the lower bound for 3-round minimum linear active S-boxes through detailed 2-round linear active S-box analysis. Specifically, we search for the minimum number of 2-round linear active S-boxes with input  $\Gamma Z_i$  and output  $\Gamma X_{i+1}$ :

$$\Gamma Z_i \xrightarrow{\pi^{-1}} \Gamma Y_i \xrightarrow{\theta} \Gamma X_{i+1}$$

For convenience of expression, we denote  $\Gamma Y_{i,j}$  as the  $j$ -th plane of  $\Gamma Y_i$ . We divide into 32 cases based on whether  $\Gamma Y_{i,j}$  is zero. Then for these 32 cases, we separately compute the number of active S-boxes for 2 rounds, as shown in

Table 4-4, where 0 indicates  $\Gamma Y_{ij}=0$ , and 1 indicates  $\Gamma Y_{ij}\neq 0$ . Due to excessive complexity, when the number of non-zero input linear mask planes in  $Y_i$  exceeds 3, we cannot compute the minimum number of worst-case linear active S-boxes for 2 rounds; using Theorem 4-2, we provide conservative lower bound values for active S-boxes.

Table 4-4: Detailed Values of 2-Round Linear Active S-Boxes

| $\Gamma Y_{i,0}, \Gamma Y_{i,1}, \Gamma Y_{i,2},$<br>$\Gamma Y_{i,3}, \Gamma Y_{i,4}$ | Active S-boxes | $\Gamma Y_{i,0}, \Gamma Y_{i,1}, \Gamma Y_{i,2},$<br>$\Gamma Y_{i,3}, \Gamma Y_{i,4}$ | Active S-boxes |
|---------------------------------------------------------------------------------------|----------------|---------------------------------------------------------------------------------------|----------------|
| 0,0,0,0,0                                                                             | 0              | 0,0,1,1,1                                                                             | 23             |
| 0,0,0,0,1                                                                             | 12             | 0,1,0,1,1                                                                             | 24             |
| 0,0,0,1,0                                                                             | 12             | 0,1,1,0,1                                                                             | 23             |
| 0,0,1,0,0                                                                             | 12             | 0,1,1,1,0                                                                             | 23             |
| 0,1,0,0,0                                                                             | 12             | 1,0,0,1,1                                                                             | 23             |
| 1,0,0,0,0                                                                             | 12             | 1,0,1,0,1                                                                             | 23             |
| 0,0,0,1,1                                                                             | 18             | 1,0,1,1,0                                                                             | 23             |
| 0,0,1,0,1                                                                             | 18             | 1,1,0,0,1                                                                             | 23             |
| 1,0,0,1,0                                                                             | 18             | 1,1,0,1,0                                                                             | 23             |
| 0,1,0,0,1                                                                             | 18             | 1,1,1,0,0                                                                             | 24             |
| 0,1,0,1,0                                                                             | 18             | 0,1,1,1,1                                                                             | $\geq 24$      |
| 0,1,1,0,0                                                                             | 18             | 1,0,1,1,1                                                                             | $\geq 23$      |
| 1,0,0,0,1                                                                             | 18             | 1,1,0,1,1                                                                             | $\geq 24$      |
| 0,0,1,1,0                                                                             | 18             | 1,1,1,0,1                                                                             | $\geq 24$      |
| 1,0,1,0,0                                                                             | 18             | 1,1,1,1,0                                                                             | $\geq 24$      |
| 1,1,0,0,0                                                                             | 18             | 1,1,1,1,1                                                                             | $\geq 24$      |

From Table 4-4, it can be seen that when the number of non-zero input linear mask planes in  $\Gamma Y_i$  exceeds 2, the minimum value of 2-round linear active S-boxes is 23.

**Theorem 4-2:** Assuming  $\Gamma Y_i$  originally has  $t$  planes with non-zero input linear mask, then after adding one more plane with non-zero input linear mask, the minimum number of 2-round linear active S-boxes will not decrease.

*Proof:* Assume  $\Gamma Y_i$  with one additional plane's input linear mask added is  $\Gamma Y'_i$ , and let  $\Gamma Z_i = \pi(\Gamma Y_i)$ ,  $\Gamma X_{i+1} = \theta(\Gamma Y_i)$ ;  $\Gamma Z'_i = \pi(\Gamma Y'_i)$ ,  $\Gamma X'_{i+1} = \theta(\Gamma Y'_i)$ . Since  $\Gamma Y'_i$

has one additional plane's input linear mask compared to  $\Gamma Y_i$ , according to the properties of  $\pi$ ,  $\Gamma Z'_i$  has additional non-zero lanes compared to  $\Gamma Z_i$ , and  $\Gamma X'_{i+1}$  has additional non-zero lanes compared to  $\Gamma X_{i+1}$ . Therefore, the number of active S-boxes for  $\Gamma Z'_i$  is no less than that for  $\Gamma Z_i$ , and the number of active S-boxes for  $\Gamma X'_{i+1}$  is no less than that for  $\Gamma X_{i+1}$ . Thus, the minimum number of 2-round linear active S-boxes will not decrease. ■

We now prove that when the number of non-zero input linear mask planes in round  $i$  is less than 3, the minimum number of linear active S-boxes over rounds  $i$ ,  $i+1$ , and  $i+2$  is at least 24. Since direct search is too time-consuming, we perform the following 2-round search:

$$\Gamma Y_i \xrightarrow{\theta} \Gamma X_{i+1} \xrightarrow{\chi^{-1}} \Gamma Z_{i+1} \xrightarrow{\pi^{-1}} \Gamma Y_{i+1}$$

We search for linear mask pairs with input  $\Gamma Y_i$  and output  $\Gamma Y_{i+1}$ . To obtain fine-grained results, we exhaustively search over all possibilities where the plane weights of  $\Gamma Y_i$  and  $\Gamma Y_{i+1}$  are 1 or 2, totaling  $15 \times 15 = 225$  cases. In the search process, we set the lower bound to stop at 23. The results show that for 224 cases the minimum linear active S-boxes are no less than 23. Only for the case where  $\Gamma Y_{i,0}||\Gamma Y_{i,1}||\Gamma Y_{i,2}||\Gamma Y_{i,3}||\Gamma Y_{i,4}=11000$  and  $\Gamma Y_{i+1,0}||\Gamma Y_{i+1,1}||\Gamma Y_{i+1,2}||\Gamma Y_{i+1,3}$

$||\Gamma Y_{i+1,4}=11000$ , the searched minimum linear active S-boxes is 20. In this case, the number of linear active S-boxes for  $\Gamma Y_i$  is no less than 10, and since the plane weight of  $\Gamma Y_{i+1}$  is 2, from Table 4-4 the number of linear active S-boxes in rounds  $i+1$  and  $i+2$  is no less than 18. Therefore, the total number over three rounds is no less than  $10+18=28$ .

From the above analysis, when the plane weight of  $\Gamma Y_i$  is at most 2 and that of  $\Gamma Y_{i+1}$  is at most 2, the number of linear active S-boxes over three consecutive rounds is at least 24.

In summary, considering the following three cases, the minimum number of linear active S-boxes over three consecutive rounds of permutation P is no less than 24.

1)When the plane weight of  $\Gamma Y_i$  is greater than 2, from Table 4-4, the number of active S-boxes in rounds  $i$  and  $i+1$  is no less than 23, so over three rounds it is no less than 24.

2)When the plane weight of  $\Gamma Y_i$  is at most 2 and the plane weight of  $\Gamma Y_{i+1}$  is greater than 2, from Table 4-4, the number of active S-boxes in rounds  $i+1$  and  $i+2$  is no less than 23, so over three rounds it is no less than 24.

3)When the plane weights of  $\Gamma Y_i$  and  $\Gamma Y_{i+1}$  are both at most 2, then the number of linear active S-boxes over three consecutive rounds is at least 24.

In fact, for the number of 3-round linear active S-boxes, the calculation of the

above 24 active S-boxes is very conservative. We performed MILP search on active S-boxes for a total of 3 rounds  $i$ ,  $i+1$ , and  $i+2$ . After multiple attempts and extended searching, we found that when  $Y_i$  has only one plane with non-zero input mask and the input and output masks of that plane are both 6, the minimum number of active S-boxes for a real 3-round path is 64, i.e., the upper bound for the number of 3-round linear active S-boxes is 64, as shown in Table 4-5. **We conjecture that 64 is very likely the lower bound for the number of 3-round linear active S-boxes.**

Table 4-5: Detailed Values of 3-Round Linear Active S-Boxes

| $\Gamma Y_{i,0}, \Gamma Y_{i,1}, \Gamma Y_{i,2}, \Gamma Y_{i,3}, \Gamma Y_{i,4}$ | Active S-boxes | Active S-boxes per Round |
|----------------------------------------------------------------------------------|----------------|--------------------------|
| 1,0,0,0,0                                                                        | 64             | 6---6---52               |
| 0,1,0,0,0                                                                        | 64             | 6---6---52               |
| 0,0,1,0,0                                                                        | 67             | 6---6---54               |
| 0,0,0,1,0                                                                        | 68             | 6---6---56               |
| 0,0,0,0,1                                                                        | 66             | 6---6---54               |

In summary, the linear active S-boxes for permutation P is shown in Table 4-6.

Table 4-6: Linear Active S-Box Count for Permutation P

| Rounds                    | 1 | 2  | 3       |
|---------------------------|---|----|---------|
| Min Linear Active S-boxes | 1 | 12 | [24,64] |

#### 4.1.2.3 Integral Analysis

The overall structure of permutation P is an SPN with width 1600 bits, using 5-bit S-boxes of algebraic degree 3; therefore, permutation P is a (5,3,320)-SPN. Todo [Tod15] provided integral distinguisher results for (5,2,320)-SPN and (5,4,320)-SPN, but did not give results for (5,3,320)-SPN. Using Algorithm 2 proposed by Todo in [Tod15], we obtained integral distinguisher results for permutation P (i.e., (5,3,320)-SPN) as shown in Table 4-7.

Table 4-7: Integral Distinguishers for Permutation P

| Rounds          | 5        | 6         | 7         | 8          | 9          | 10         | 11         |
|-----------------|----------|-----------|-----------|------------|------------|------------|------------|
| Data Complexity | $2^{84}$ | $2^{245}$ | $2^{730}$ | $2^{1310}$ | $2^{1505}$ | $2^{1570}$ | $2^{1590}$ |

From Table 4-7, it can be seen that the integral distinguisher for permutation P has 11 rounds, with data complexity  $2^{1590}$ . Considering the actual security of Laurus algorithm ( $2^{1024}$ ), the distinguisher rounds can only reach 7. If more detailed characterization of S-boxes and linear transformation is

performed, integral distinguishers with more rounds might be obtained; however, considering the number of rounds of permutation P and actual security, Laurus algorithm can resist integral attacks

#### 4.1.2.4 Impossible Differential Analysis

$$X_0 \xrightarrow{\chi} Y_0 \xrightarrow{\theta} Z_0 \xrightarrow{\pi} X_1 \xrightarrow{\chi} \dots X_{r-1} \xrightarrow{\chi} Y_{r-1} \xrightarrow{\theta} Z_{r-1} \xrightarrow{\pi} X_r$$

For any  $r$ -round differential path above, since  $Y_i$  and  $X_{i+1}$  have a one-to-one correspondence, for efficient search, we only need to find one  $r$ -round impossible differential path from  $X_0$  to  $Y_{r-1}$  to find an  $r$ -round impossible differential path from  $X_0$  to  $X_r$ .

Through computer search, we found a 4-round impossible differential path with input difference  $X_0=1||0^{1599}$  and output  $Y_3=1||0^{1599}$ . The value of  $X_4$  at this point is shown in Table 4-8.

Table 4-8:  $X_4$  Values for Impossible Differential Path

|                                                                                           |
|-------------------------------------------------------------------------------------------|
| 8000000140000000, 0000000000000000, 0000000000000000, 0000000000000000, 0000000000000000, |
| 0000000000000000, 0000000000000000, 0000020040000000, 0000000000000000, 0000000000000000, |
| 0000000000000000, 0000000000000000, 0000000000000000, 0000000000000000, 0000040040000000, |
| 0000000000000000, 0800000040000000, 0000000000000000, 0000000000000000, 0000000000000000, |
| 0000000000000000, 0000000000000000, 0000000000000000, 4000000040000000, 0000000000000000. |

Due to computational complexity, we did not exhaust all differential paths, so longer-round impossible differential paths may exist. However, since the algorithm has sufficient round redundancy, we believe permutation P can resist impossible differential attacks.

#### 4.1.2.5 Zero-Correlation Linear Attack

$$X_0 \xrightarrow{\chi^{-1}} Z_0 \xrightarrow{\pi^{-1}} Y_0 \xrightarrow{\theta} X_1 \xrightarrow{\chi^{-1}} \dots X_{r-1} \xrightarrow{\chi^{-1}} Z_{r-1} \xrightarrow{\pi^{-1}} Y_{r-1} \xrightarrow{\theta} X_r$$

For any  $r$ -round linear path above, since  $Y_i$  and  $X_{i+1}$  have a one-to-one correspondence, for efficient search, we only need to find one  $r$ -round zero-correlation path from  $X_0$  to  $Y_{r-1}$  to find an  $r$ -round zero-correlation path from  $X_0$  to  $X_r$ .

Through computer search, we found a 4-round zero-correlation path with  $X_0=1||0^{1599}$  and output  $Y_3=1||0^{1599}$ . The value of  $X_4$  at this point is shown in Table 4-9.

Due to computational complexity, we did not exhaust all zero-correlation paths, so longer-round zero-correlation paths may exist. However, since the algorithm has sufficient round redundancy, we believe permutation P can resist zero-correlation attacks.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 80000000140000000, 40000000400000000, 08000000400000000, 00000400400000000, 00000200400000000,<br>00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000,<br>00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000,<br>00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000,<br>00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000,<br>00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000, 00000000000000000。 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

The low algebraic degree of the S-box allows the construction of internal and external zero-sum distinguishers on the permutation, preventing it from being treated as an ideal random permutation. The algebraic degree of the round function  $f$  of permutation  $P$  is 3. The upper bound on algebraic degree after  $R$  rounds is shown in Table 4-10 (estimated via Theorem 2 of [BC13]).

|                |      |      |      |      |      |      |      |      |
|----------------|------|------|------|------|------|------|------|------|
| Rounds R       | 1    | 2    | 3    | 4    | 5    | 6    | 7    | 8    |
| $\deg(f^R)$    | 3    | 9    | 27   | 81   | 243  | 729  | 1309 | 1503 |
| $\deg(f^{-R})$ | 3    | 9    | 27   | 81   | 243  | 729  | 1309 | 1503 |
| Rounds R       | 9    | 10   | 11   | 12   | 13   | 14   | 15   | 16   |
| $\deg(f^R)$    | 1567 | 1589 | 1596 | 1598 | 1599 | 1599 | 1599 | 1599 |
| $\deg(f^{-R})$ | 1567 | 1589 | 1596 | 1598 | 1599 | 1599 | 1599 | 1599 |

#### 4.1.2.7 Resistance Against Known Quantum Attacks

24

If considering the specific structure of permutation P, by utilizing its differential, linear, and other characteristics combined with Grover search and other quantum algorithms, quantum differential, linear, and other attacks can be performed, formulating quantum variants of these traditional analysis schemes. However, these quantum analysis methods merely replace the search portion of traditional analysis schemes with Grover's algorithm or QAA and other quantum algorithms; their attack strength compared to corresponding classical analysis schemes can achieve at most quadratic speedup. Since permutation P of Laurus algorithm was designed with full consideration of resistance against classical differential, linear, and other attack methods, under current analysis, the attack complexity of corresponding quantum attack schemes is higher than that of generic quantum attacks.

## 4.2 Concrete Security

The Laurus algorithm, denoted according to its output hash length as Laurus-512, Laurus-768, and Laurus-1024, has the following security results:

Table 4-11 Security Results of the Laurus Algorithm

| Algorithm   | Output Size | Classical Security Strengths in Bits |                 |                  | Quantum Security Strengths in Bits |                  |                    |
|-------------|-------------|--------------------------------------|-----------------|------------------|------------------------------------|------------------|--------------------|
|             |             | Preimage                             | Second Preimage | Collision        | Preimage                           | Second Preimage  | Collision          |
| Laurus-512  | 512         | 512                                  | 512             | 256              | 256                                | 256              | 512/3              |
| Laurus-768  | 768         | 768                                  | 768             | 384              | 384                                | 384              | 256                |
| Laurus-1024 | 1024        | 1024                                 | 1024            | 512              | 512                                | 512              | 1024/3             |
| Laurus-XOF  | $d$         | $\min(d, 512)$                       | $\min(d, 512)$  | $\min(d/2, 256)$ | $\min(d/2, 256)$                   | $\min(d/2, 256)$ | $\min(d/3, 512/3)$ |

For classical computational security, the security strengths of the four algorithms are given according to the security bounds in Section 4.1.1 “Security Analysis of the Mode of Operation.”

For quantum attacks, we have not found any practical attack that outperforms the generic quantum complexity for hash algorithms. The security strengths of the four algorithms are given according to the generic quantum security bounds for hash algorithms.

## 4.3 Other Security Properties

### 4.3.1 Statistical Test Analysis

Statistical randomness testing primarily examines the randomness of the digest stream. The tests were performed on the digest stream generated by the Laurus-1024 algorithm. The significance level for randomness testing was set to  $\alpha=0.01$ . The test items include: frequency test, serial test (also known as the

poker test in some contexts), autocorrelation test, and runs test (5 items in total). Message samples were generated using five different methods: random message generation, counter generation, output feedback generation, low Hamming weight generation, and high Hamming weight generation. Under each message sample generation method,  $2^{20}$  groups of 512-bit message samples were produced, and the message digest of each sample was calculated and concatenated into a digest stream. Under each generation method, a total of  $2^{30}$  bits of digest stream were obtained, which were then divided into 1024 groups, each of length  $2^{20}$  bits. With the significance level  $\alpha=0.01$ , the number of passing groups for each randomness test should be no less than 1005 ( $\lceil 1024 \times (1 - \alpha - 3 \times (\alpha \times (1 - \alpha) / 1024)^{1/2}) \rceil = 1005$ ).

The test results below demonstrate that the Laurus algorithm passes the statistical randomness tests.

Table 4-12 Random Message Generation Method

| Test No. | Test Item       | Test Parameters | Test Groups | Passing Groups |
|----------|-----------------|-----------------|-------------|----------------|
| 1        | Frequency Test  | None            | 1024        | 1020           |
| 2        | Serial Test     | None            | 1024        | 1012           |
| 3        | Poker Test      | 4               | 1024        | 1012           |
|          |                 | 8               | 1024        | 1016           |
| 4        | Autocorrelation | 1               | 1024        | 1008           |
|          |                 | 2               | 1024        | 1020           |
|          |                 | 8               | 1024        | 1008           |
|          |                 | 16              | 1024        | 1008           |
| 5        | Runs Test       | None            | 1024        | 1008           |

Table 4-13 Counter Generation Method

| Test No. | Test Item       | Test Parameters | Test Groups | Passing Groups |
|----------|-----------------|-----------------|-------------|----------------|
| 1        | Frequency Test  | None            | 1024        | 1007           |
| 2        | Serial Test     | None            | 1024        | 1012           |
| 3        | Poker Test      | 4               | 1024        | 1016           |
|          |                 | 8               | 1024        | 1010           |
| 4        | Autocorrelation | 1               | 1024        | 1014           |
|          |                 | 2               | 1024        | 1018           |
|          |                 | 8               | 1024        | 1016           |
|          |                 | 16              | 1024        | 1015           |

|   |           |      |      |      |
|---|-----------|------|------|------|
| 5 | Runs Test | None | 1024 | 1014 |
|---|-----------|------|------|------|

Table 4-14 Output Feedback Generation Method

| Test No. | Test Item       | Test Parameters | Test Groups | Passing Groups |
|----------|-----------------|-----------------|-------------|----------------|
| 1        | Frequency Test  | None            | 1024        | 1011           |
| 2        | Serial Test     | None            | 1024        | 1014           |
| 3        | Poker Test      | 4               | 1024        | 1018           |
|          |                 | 8               | 1024        | 1013           |
| 4        | Autocorrelation | 1               | 1024        | 1011           |
|          |                 | 2               | 1024        | 1016           |
|          |                 | 8               | 1024        | 1021           |
|          |                 | 16              | 1024        | 1017           |
| 5        | Runs Test       | None            | 1024        | 1011           |

Table 4-15 Low Hamming Weight Generation Method

| Test No. | Test Item       | Test Parameters | Test Groups | Passing Groups |
|----------|-----------------|-----------------|-------------|----------------|
| 1        | Frequency Test  | None            | 1024        | 1013           |
| 2        | Serial Test     | None            | 1024        | 1015           |
| 3        | Poker Test      | 4               | 1024        | 1014           |
|          |                 | 8               | 1024        | 1015           |
| 4        | Autocorrelation | 1               | 1024        | 1013           |
|          |                 | 2               | 1024        | 1018           |
|          |                 | 8               | 1024        | 1011           |
|          |                 | 16              | 1024        | 1011           |
| 5        | Runs Test       | None            | 1024        | 1013           |

Table 4-16 High Hamming Weight Generation Method

| Test No. | Test Item       | Test Parameters | Test Groups | Passing Groups |
|----------|-----------------|-----------------|-------------|----------------|
| 1        | Frequency Test  | None            | 1024        | 1010           |
| 2        | Serial Test     | None            | 1024        | 1018           |
| 3        | Poker Test      | 4               | 1024        | 1017           |
|          |                 | 8               | 1024        | 1011           |
| 4        | Autocorrelation | 1               | 1024        | 1014           |
|          |                 | 2               | 1024        | 1019           |

|   |           |      |      |      |
|---|-----------|------|------|------|
|   |           | 8    | 1024 | 1018 |
|   |           | 16   | 1024 | 1008 |
| 5 | Runs Test | None | 1024 | 1015 |

### 4.3.2 Length Extension Attacks

A length extension attack means the following: for an unknown message  $M$ , given its hash value  $H(M)$ , does there exist a bit string  $x$  such that  $H(M||x)$  can be computed. This attack is effective against early hash algorithms based on the MD structure, because they often output the complete final chaining value. By appropriately adjusting the padding process and computing the compression function in sequence, a new hash value can be successfully constructed.

For the mode of operation used by the Laurus algorithm, as can be seen from Figure 2-1, the first bit of the counter in the message processing phase is fixed to 0, and the first bit of the counter in the hash output phase is fixed to 1. Therefore, these two phases can be viewed as two different functions. Thus, when the number of padded message blocks of  $M||x$  is different from that of  $M$ ,  $H(M)$  cannot be an intermediate state in the computation of  $H(M||x)$ . If the number of padded message blocks of  $M||x$  is the same as that of  $M$ , then using  $H(M)$  to compute  $H(M||x)$  would require inverting the final output phase of  $H(M)$ , which is infeasible.

In summary, the Laurus algorithm is resistant to length extension attacks.

## 5 Performance Evaluation

### 5.1 Performance Analysis

#### 5.1.1 Design Efficiency of the Overall Structure

##### (1) Software-Hardware Compatibility of the Mode of Operation

The mode of operation adopted by Laurus unifies message compression and hash value output into iterations of the same permutation function  $P$ , effectively improving implementation efficiency in both software and hardware.

- **Software level:** The cyclic iteration approach facilitates implementation as compact loop code; the state can be stored contiguously in registers or memory, reducing function call overhead. The symmetry between message compression and hash output phases reduces code branching complexity.
- **Hardware level:** A unified permutation function means that only one instance of the  $P$  circuit needs to be instantiated; the entire computation can be completed through feedback reuse, achieving high resource utilization. The XOR operations in the compression phase alternate

serially with the permutation, resulting in simple control logic.

## (2) Selection of State Size

The 1600-bit state is a core parameter of the algorithm; this size falls within the typical range of existing hash algorithms, balancing security strength and implementation efficiency. For software implementation efficiency, 1600 bits can be divided into 25 64-bit words or 50 32-bit words, adapting to register widths of mainstream processors. On 64-bit platforms, the state can be fully loaded into general-purpose registers, reducing memory access. For hardware implementation efficiency, 1600-bit registers have controllable area in FPGA/ASIC.

### 5.1.2 Software and Hardware Efficiency of Core Components

#### (1) Combinational Logic Characteristics of Permutation Function P

Permutation function P is designed as pure combinational logic with no internal state storage. This characteristic demonstrates efficiency advantages at both software and hardware levels.

- **Software level:** Pure combinational logic translates to a series of bitwise operations (XOR, shift, AND, OR, etc.) with no conditional branches, facilitating compiler optimization and easy parallelization using SIMD instructions.
- **Hardware level:** Combinational logic circuits have no clock dependency; complete transformation is accomplished in a single cycle; critical path length is determined only by gate-level delay, suitable for high-frequency operation.

#### (2) Efficiency of Bit-Sliced S-box Implementation

The 5-bit S-boxes adopted by this algorithm have logic that can be directly converted to bitwise operations, avoiding lookup table overhead. During bit-sliced implementation, the 1600-bit state can be decomposed into parallel processing of 25 64-bit states. This design achieves high parallelism in both software and hardware.

## 5.2 Performance Testing

### 5.2.1 Software Performance Testing

#### (1) Detailed Configuration of Implementation Platform

Laurus hash algorithm was tested and evaluated on a 64-bit platform using Intel Core i3-6100 CPU. Platform specifications are shown in Table 5-1.

Table 5-1 Software Evaluation Platform

| CPU                                                | Memory | Operating System | Programming Language | Compiler | Instruction Set | Cryptographic Library |
|----------------------------------------------------|--------|------------------|----------------------|----------|-----------------|-----------------------|
| Intel(R)<br>Core(TM)<br>i3-6100<br>CPU<br>@3.70GHz | 16GB   | Win7-64          | Standard C           | VS2022   | None            | None                  |

## (2) Implementation Methods

① Optimized implementation of nonlinear transformation: According to the scheme description, nonlinear transformation requires 21 slice operations.

$$B[0] = A[0] \& A[2] \& A[3] \oplus A[2] \& A[3] \& A[4] \oplus A[0] \& A[4] \oplus A[3] \& A[4] \oplus A[1];$$

$$B[1] = A[1] \& A[3] \& A[4] \oplus A[0] \& A[1] \oplus A[2];$$

$$B[2] = A[1] \& A[2] \oplus A[3];$$

$$B[3] = A[2] \& A[3] \oplus A[4];$$

$$B[4] = A[3] \& A[4] \oplus A[0].$$

By reusing partial operations, the number of operations is reduced:

$$B[4] = A[3] \& A[4] \oplus A[0];$$

$$B[3] = A[2] \& A[3] \oplus A[4];$$

$$B[2] = A[1] \& A[2] \oplus A[3];$$

$$B[1] = B[4] \& A[1] \oplus A[2];$$

$$B[0] = B[4] \& B[3] \oplus A[1].$$

It can be seen that only 10 slice operations are needed, reducing operations by more than half.

② Optimized implementation of linear transformation and transposition transformation: Linear transformation and transposition transformation are combined to reduce transposition operations.

$$T[i][j] = A[i][j] \oplus (E_i \ggg t_{i,j}) \oplus F_i$$

$$B[i][j] = T[(i + 2j) \bmod 5][(3i + 3j) \bmod 5]$$

Change to:

$$B[(4i + 4j) \bmod 5][(i + 3j) \bmod 5] = A[i][j] \oplus (E_i \ggg t_{i,j}) \oplus F_i$$

③ Addition of message update function to improve code flexibility.

④ Unification of absorption/release phases for the four function types (XOF, 512, 768, 1024) into a single function stage, reducing code redundancy while facilitating code management.

⑤ Other optimization methods: Macro-definition of low-level small module functions to reduce code call paths.

### (3) Performance Test Results

Table 5-2 Operation Speed of Laurus-512/768/1024 vs. SHA3-512

| Level | Data Length<br>(Bytes) | speed(Mbps) |            |             |          |
|-------|------------------------|-------------|------------|-------------|----------|
|       |                        | Laurus-512  | Laurus-768 | Laurus-1024 | SHA3-512 |
| S1    | 32                     | 411.3       | 296.6      | 410.1       | 370.6    |
| S2    | 128                    | 890.0       | 776.3      | 865.0       | 821.9    |
| S3    | 512                    | 1446.2      | 1118.7     | 913.2       | 839.8    |
| S4    | 1024                   | 1620.9      | 1330.0     | 920.7       | 905.7    |
| S5    | 4096                   | 1763.7      | 1347.6     | 922.1       | 957.7    |
| S6    | 8192                   | 1779.5      | 1373.2     | 929.0       | 959.9    |
| S7    | 16384                  | 1788.3      | 1382.1     | 926.8       | 965.0    |
| S8    | 65536                  | 1812.0      | 1382.2     | 932.4       | 968.1    |

### 5.2.2 Hardware Performance Testing

The Laurus hash algorithm outputs hash values of 512 bits, 768 bits, and 1024 bits, corresponding to three hardware implementation versions. All three versions were performance tested on both FPGA and ASIC.

#### (1) Detailed Configuration of Implementation Platform

Laurus hash algorithm was implemented and analyzed on xc7z100ffg900-2 FPGA chip and SMIC 28nm chip. Modelsim SE 10.1c was used for simulation. For FPGA testing, synthesis tool Vivado 2018.3 was used; for ASIC testing, Synopsys Design Compiler tool version syn\_v0\_2018.06-sp1 was used.

#### (2) Implementation Methods

Hardware implementation adopts a top-down approach; the algorithm top level contains only signal interconnections, with modules communicating via control and data signals. The hardware circuit logic block diagram is shown in Figure 5-1.

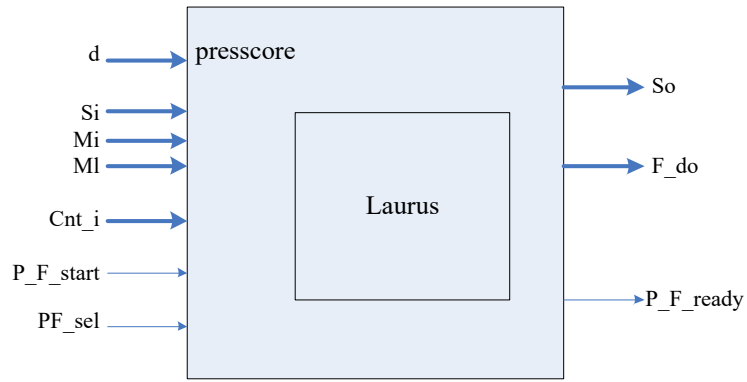


Figure 5-1 Hardware Circuit Logic Block Diagram

Logic top-level signal definitions are shown in Table 5-3:

Table 5-3 Logic Top-Level Signal Definitions

| Signal Name      | Signal Definition                                                        |
|------------------|--------------------------------------------------------------------------|
| <i>d</i>         | 1-bit auxiliary variable input for hash output phase                     |
| <i>Si</i>        | Iterative state input                                                    |
| <i>Mi</i>        | Message block input                                                      |
| <i>Ml</i>        | Final message block input                                                |
| <i>Cnt_i</i>     | Counter                                                                  |
| <i>P_F_start</i> | Processing start signal                                                  |
| <i>P_F_ready</i> | Processing completion signal                                             |
| <i>PF_sel</i>    | Function selection: 0 = message compression phase, 1 = hash output phase |
| <i>So</i>        | Iterative state output                                                   |
| <i>F_do</i>      | Hash result output                                                       |

### (3) Performance Test Results

FPGA test results are shown in Table 5-4:

Table 5-4 FPGA Test Results

| Version     | Resource usage |      |      | Performance simulation results |                         |                       |
|-------------|----------------|------|------|--------------------------------|-------------------------|-----------------------|
|             | Flip Flops     | LUTs | BRAM | Clock Period (ns)              | Short Packet 64B (Gbps) | Long Packet 1MB(Gbps) |
| Laurus-512  | 1670           | 6365 | 0    | 2.9                            | 10.27                   | 20.55                 |
| Laurus-768  | 1670           | 6689 | 0    | 2.9                            | 10.27                   | 15.41                 |
| Laurus-1024 | 1670           | 6620 | 0    | 3.0                            | 9.93                    | 9.93                  |

ASIC test results are shown in Table 5-5:

Table 5-5: ASIC Test Results

| Version     | Resource usage (GE) | Performance simulation results |                            |                          |
|-------------|---------------------|--------------------------------|----------------------------|--------------------------|
|             |                     | Clock Period (ns)              | Short Packet<br>64B (Gbps) | Long Packet<br>1MB(Gbps) |
| Laurus-512  | 40962               | 0.3                            | 99.34                      | 198.65                   |
| Laurus-768  | 42040               | 0.3                            | 99.34                      | 149.01                   |
| Laurus-1024 | 43593               | 0.3                            | 99.34                      | 99.34                    |

## 6 Features

### 6.1 Innovativeness

#### (1) Innovative design of a secure and efficient mode of operation

We have innovatively designed a mode of operation for the hash algorithm. Compared with the widely used Sponge structure, it achieves stronger indistinguishability security and higher message compression efficiency. For example, for Laurus-512, the message compression efficiency of its mode of operation can reach twice that of SHA3-512.

#### (2) Innovative design of a simple and highly secure linear transformation

We have designed a parameterized 320-bit linear transformation, Theta, whose implementation depth is the same as that of Keccak's linear transformation, but whose branch number is increased from 4 to 12. A software implementation requires only 18 64-bit XOR operations and 10 cyclic shifts. By varying the parameters, different linear transformations can be constructed. Based on this linear transformation, we have designed a secure and simple 1600-bit internal permutation.

#### (3) Innovative design of a secure and simple internal permutation

The Laurus algorithm uses an internal permutation of 1600 bits, iterated over 16 rounds. All components of the round function are simple and efficient, and all operations can be processed in parallel. For example, the nonlinear transformation consists of 5-bit S-boxes; in a bitsliced implementation, the 1600-bit state is decomposed into 25 64-bit lanes for parallel processing. The transposition transformation incurs no implementation cost in either hardware or software. Moreover, the components of the internal permutation are closely interrelated, achieving high security. For example, full diffusion is achieved in 3 rounds, and the number of differentially active S-boxes over 3 rounds likely reaches around 60, which is a significant improvement over the round function of the Keccak permutation in terms of resistance to differential and linear attacks.

### 6.2 Simplicity

### (1) Simplicity of the mode of operation

Similar to the Sponge construction, the mode of operation is based on the internal permutation  $P$ , with other operations limited to simple XORs and state updates. There are no complex operations such as message expansion.

### (2) Simplicity of the internal permutation

All components of the internal permutation involve only bitwise AND, XOR, and cyclic shift operations, and each component is very simple, requiring few operations and having low depth.

## 6.3 Flexibility

### (1) Support for different hash output lengths

Laurus-512, Laurus-768, and Laurus-1024 share the same internal permutation  $P$ ; by adjusting parameters, different output lengths can be produced.

### (2) Ease of constructing XOF and MAC

Like Keccak, Laurus supports extendable output lengths, and a MAC can be readily constructed by concatenating a key with the message.

### (3) Excellent parallelism

The parallelism of the algorithm is mainly reflected in the following aspects:

- ① The nonlinear transformation supports parallel implementation.
- ② The linear transformation supports parallel implementation.
- ③ The extendable-output function (XOF) supports parallel output generation.

## 6.4 Implementation Performance Advantages on Different Platforms

### (1) Efficient software and hardware implementation

All operations of the Laurus algorithm are based on 64-bit words. Each component of the internal permutation  $P$  is simple and efficient. A single S-box requires only 10 basic operations; the linear transformation  $\theta_i$  requires only 18 64-bit XORs and 10 cyclic shifts; the transposition transformation incurs no additional implementation cost.

### (2) Excellent parallel implementation

The mode of operation supports parallel output when used as an extendable-output function (XOF), and the main cryptographic components of the internal permutation all support parallel processing.

## 7 Comparison with SHA-3 Algorithm

Both Laurus-512 and SHA3-512 support a hash output of 512 bits, and both use an internal permutation state of 1600 bits. Based on the same platform and with a comparable level of optimization, we implemented both algorithms in software and hardware. Table 7-1 provides a comparison of the two algorithms in terms of security and efficiency.

Table 7-1 Comparison of Laurus-512 and SHA3-512

| Algorithm  | Collision Resistance | Preimage Resistance | Second Preimage Resistance | Indifferentiability | Software Efficiency | Hardware Efficiency |
|------------|----------------------|---------------------|----------------------------|---------------------|---------------------|---------------------|
| Laurus-512 | $O(2^{256})$         | $O(2^{512})$        | $O(2^{512})$               | $O(2^{512})$        | 1812.0Mbps          | 20.55Gbps           |
| SHA3-512   | $O(2^{256})$         | $O(2^{512})$        | $O(2^{512})$               | $O(2^{512})$        | 968.1Mbps           | 8.94Gbps            |

*Note: The software platform is an Intel Core i3-6100 CPU, Windows OS. The hardware efficiency figures are from FPGA implementations using Vivado 2018.3 on an xc7z100ffg900-2 chip.*

## 8 References

- [BC13] C.Boura and A.Canteaut, “On the influence of the algebraic degree of  $F^{-1}$  on the algebraic degree of GF, in IEEE Transactions on Information Theory, vol.59, no.1, pp.691-702,Jan.2013.
- [BD06] Eli Biham and Orr Dunkelman. A framework for iterative hash functions–HAIFA. Second NIST Cryptographic Hash Workshop, 2006, 2006.
- [BDPV07] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche.“Sponge functions”. Ecrypt Hash Workshop 2007. 2007.
- [BDPV08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche.“On the Indifferentiability of the Sponge Construction”. In: EUROCRYPT2008. Vol. 4965. LNCS. Springer, 2008.
- [BDPV11] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche.“The Keccak Reference”. Submission to NIST (Round 3).
- [Dam90] Ivan Damgård. A design principle for hash functions. In Gilles Brassard,editor, CRYPTO'89, volume 435 of LNCS, pages 416-427. Springer, New York,

August 1990.

- [DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schl  ffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *J. Cryptol.*, 34(3):33, 2021.
- [DM15] Dworkin, M.J., et al.: FIPS PUB 202-- SHA-3 standard: Permutation-based hash and extendable output functions (2015).
- [HWL24] Lei H, Wang W, Zhang L. Symmetric twin column parity mixers and their applications[J]. *IACR Transactions on Symmetric Cryptology*, 2024, 2024(1).
- [LLHL26] Hongli Li, Changlun Li, Honggang Hu, and Fengmei Liu. Indifferentiability of the sponge hash family: New attacks and the optimal construction. *IACR Transactions on Symmetric Cryptology*, 2026(1):31-75, 2026.
- [Luc05] Stefan Lucks. A failure-friendly design principle for hash functions. In Bimal K. Roy, editor, *ASIACRYPT*, volume 3788 of *Lecture Notes in Computer Science*, pages 474-494. Springer, 2005.
- [Mer90] Ralph C. Merkle. One way hash functions and DES. In Gilles Brassard, editor, *CRYPTO'89*, volume 435 of *LNCS*, pages 428-446. Springer, New York, August 1990.
- [NP10] Mridul Nandi and Souradyuti Paul. Speeding up the wide-pipe: Secure and fast hashing. In Guang Gong and Kishan Chand Gupta, editors, *INDOCRYPT2010*, volume 6498 of *LNCS*, pages 144-162. Springer, Berlin, Heidelberg, December 2010.
- [Tod15] Yosuke Todo. "Structural Evaluation by Generalized Integral Property". In: *EUROCRYPT 2015*. Vol. 9056. LNCS. Springer, 2015, pp. 287-314.